Тренажёр оператора ДДС города Москвы

Сопроводительная документация

Задача №9 Департамента по делам гражданской обороны, чрезвычайным ситуациям и пожарной безопасности города Москвы. Лидеры цифровой трансформации, 2026.

---

1. Назначение

Тренажёр готовит операторов дежурно-диспетчерских служб и Системы-112 к работе в боевой системе **ПОВ-112 (АРМ-112)**. Оба участка из ТЗ отрабатываются в интерфейсе, повторяющем боевой:

- **оператор 112** принимает учебный вызов: выясняет у заявителя обстоятельства, заполняет карточку происшествия, классифицирует его и оповещает службы;
- **диспетчер ДДС** получает поток карточек своей службы: принимает или мотивированно отклоняет, назначает бригаду, по докладам старшего ведёт статусы реагирования до завершения.

Преподаватель проводит занятия по группе, готовит учебные вызовы и видит аналитику. После каждого вызова курсант получает разбор с ценой каждой ошибки, прогноз своего результата и рекомендации — и то, и другое система потом проверяет фактом. Свободный текст курсанта читает модель и указывает расхождения с услышанным, с цитатами; преподаватель подтверждает их и ставит экспертную оценку рядом с автоматической.

Ключевая особенность решения — **тренажёр отличает выясненное от угаданного**. Раздел 3.3 объясняет, почему это главная метрика подготовки оператора.

---

2. Состав решения

Компонент	Технология	Назначение
Веб-клиент	React 19, TypeScript, Vite	интерфейс АРМ-112 и учебный слой
Сервер приложения	Rust, axum, tokio	API, домен, оценка, фоновые задачи
Хранилище	PostgreSQL 16	классификатор, вызовы, занятия, журналы, настройки
Языковая модель	GigaChat 3.1 10B-A1.8B на llama.cpp	подготовка учебных вызовов, правка по переписке, сводки, анализ текста курсанта
Голос	GigaAM v3 (распознавание) и TeraTTSv2 (синтез), ONNX на процессоре	разговор диспетчера ДДС со старшим бригады голосом

Компонент	Технология	Назначение
Сверка описания	кросс-энкодер OpenJEV (Qwen3.5, 2B или 4B), PyTorch на процессоре	сверка описания курсанта с фактами вызова, раздел 3.9

Все шесть компонентов разворачиваются на одной машине без выхода в интернет и без графического ускорителя — одной командой, раздел 7.7. Обоснование выбора — раздел 6.

Объём исходного кода: 15 848 строк Rust в 63 файлах, 5 673 строки TypeScript в 49 файлах, 28 миграций схемы базы.

---

3. Методы обработки данных

3.1. Классификация происшествия — детерминированная, без нейросети

Единый классификатор происшествий (ЕКП) — это таблица, где каждой комбинации признаков соответствует **ровно один** правильный ответ. Поэтому классификацию считает не модель, а прямой поиск по таблице:

```
ответы карты ──①──► признаки ──②──► позиция ЕКП ──③──► список оповещения
```

① Ответы опросной карты сопоставляются с признаками трёх уровней («где», «что именно», «уточнение»). ② По группе и признакам ищется позиция классификатора. Если незаполненный уровень признаков оставляет несколько позиций с разными исходами, система **не выбирает первую попавшуюся**, а называет варианты и просит уточнить — так же ведёт себя боевая система. ③ Позиция задаёт список оповещения. Службы, получающие лишь копию карточки (значения «карточка-112», «нет реагирования»), отделяются от реагирующих: оценка не должна штрафовать за то, что оператор их не вызвал.

В активной версии классификатора **1 283 позиции в 24 группах**, **134 593 строки матрицы оповещения**, 60 служб.

Такой выбор даёт воспроизводимость: эталон для учебного вызова не зависит от того, какая модель загружена и загружена ли вообще.

3.2. Опросные карты для всех типов происшествия

В выпадающем списке «Что случилось?» — **51 тип**, как в боевой системе (полный список передан экспертами в чате задачи). Опросная карта есть у **44** из них:

- **3 карты сняты с кадров боевой системы** (101 — пожар, 104 — газ, «Взрыв»): формулировки вопросов и вариантов дословные. Карта 101 снята по трём веткам — «Дом», «Улица» и «Транспорт»;
- **41 карта выведена из классификатора**: опросная карта боевой системы — это дерево признаков группы, и классификатор содержит его целиком. Подписи вопросов — заголовки столбцов классификатора, варианты — дословно из ячеек. Ничего не сочинено;

- **у 7 типов карты нет** («Внутренний звонок», «Нецелевой вызов», «Технический сбой» и др.): это виды обращений, а не происшествия, в классификаторе им не соответствует ни один признак. Интерфейс так и пишет, а не показывает пустую форму.

Карта **растёт по мере ответов**, как на кадрах боевой системы:

- варианты второго признака зависят от первого (`depends_on`): у «ДТП» одни, у «ДТП с пострадавшими» — другие; смена первого признака снимает неподходящие ответы на втором;
- вопрос целиком появляется только при нужном ответе (`show_when`): на пустом «Где» у пожара видны только «Где» и «Описание», после «Улица» — «Признак пожара», «Доступ», «Место происшествия», а «Описание правонарушения» — лишь после «Есть правонарушение».

Карты выводятся при каждом запуске сервера и при активации новой версии классификатора — поэтому смена версии не оставляет устаревших карт.

3.3. Оценка оператора 112 — три слоя

Разбор сравнивает три независимых слоя данных:

Слой	Что это
Скрытая истина	что на самом деле произошло, заложено в сценарий
Стенограмма	что заявитель действительно сообщил в разговоре
Карточка	что оператор ввёл в поля

Из сопоставления рождаются четыре вида замечаний с разным весом:

Вид	Вес	Смысл
`guessed`	1	значение верное, но заявитель его не сообщал — **угадал**
`extra`	1	вызвана лишняя служба
`missing`	2	служба не оповещена
`wrong`	3	значение не совпало с эталоном

Превышение норматива времени добавляет 2.

Замечание формулируется как последствие, а не как балл: не «−3 за поле „служба“», а «служба не оповещена — фактически не выехала».

Смысл метрики «угадал»: оператор, угадавший тип происшествия по первой фразе, в боевой обстановке ошибётся, как только вызов окажется нетипичным. На данных стенда доля угаданного составила **38 %** — то есть больше трети верных ответов курсанты не выясняли у заявителя.

3.4. Работа диспетчера ДДС: статусы, нормативы, бригада

Статусы реагирования идут строго по памятке ДДС: сначала «Принята» или «Не принята», затем «Начало реагирования» → «Прибытие» → «Проведение работ» → «Работы завершены» или «Отказ от выполнения работ». Порядок проверяется на сервере, а не только в интерфейсе. «Не принята» и

«Отказ» без комментария сервер не принимает: заказчик требует мотивированного отказа. Поля, которые заполнил оператор 112, диспетчер ДДС не редактирует — только свои статусы и комментарии к ним.

Нормативы — по ответу заказчика в чате задачи (27.09.2026):

Норматив	Значение	Отсчёт
Открыть карточку	30 с	с момента появления карточки в строке сообщений
Первая запись: статус и текст	3 мин	см. ниже

Открытие карточки фиксируется системной отметкой «Получена службой». Точка отсчёта трёх минут в ответе заказчика («3 минуты на открытие карточки и добавление первой записи») однозначно не закреплена; мы считаем от поступления карточки. Остальные временные показатели заказчик не определял: работы могут идти часами и днями.

Статус карточки не хранится как состояние конечного автомата, а **выводится** из состояния служб и прошедшего времени. Фоновая задача (тик 5 секунд) пересчитывает статусы и переводит карточки в «Не оповещено» (норматив открытия или первой записи истёк) и «Не завершено» (48 часов, памятка ДДС). Запись условная — по прежнему статусу, поэтому два экземпляра сервера не запишут переход дважды.

Бригада. По ответам заказчика бригаду диспетчер выбирает вручную, а сведения о ходе работ получает от старшего: тот звонит с докладом сам, или диспетчер сам звонит ему. В тренажёре живут оба варианта:

- этапы работ бригады наступают по шкале от момента назначения — это скрытая истина, как факты заявителя у оператора 112;
- старший докладывает о наступившем этапе с задержкой; диспетчер, который позвонил раньше, узнаёт раньше. Доклад называет только этап («прибыли на место») — обстановку на месте мы не придумываем.

Разбор диспетчера ДДС проверяет требование памятки «статусы по факту получения информации»:

Замечание	Вес	Смысл
статус раньше доклада	2	«не по факту» — аналог «угадал» у оператора 112
доклад без статуса	2	служба 112 и соседние ДДС не видят хода работ
принята, бригада не назначена	2	на место никто не выехал
статус позже доклада сверх норматива	1	в карточке устаревшая картина
карточка открыта позже 30 с	1	норматив заказчика
первая запись позже 3 мин или без текста	1	норматив заказчика

Что здесь — наше допущение, а не норматив заказчика. Число учебных бригад (3), шкала этапов (выезд 20 с, путь 40 с, начало работ 20 с, работы 60 с), задержка доклада (25 с) и норматив обновления статуса после доклада (30 с) заказчик не называл. Шкала сжата под учебное занятие, одна

на все службы — заказчик сам попросил «универсальный алгоритм для всех». Всё вынесено в настройки и перечислено в `PROVENANCE.md` («Наши допущения») и `OPEN-QUESTIONS.md` (вопрос 18).

Доклад вышестоящему начальнику. Второй собеседник диспетчера ДДС по ответу заказчика (27.09): «голосовое общение ДДС с вышестоящим начальником и ДДС с руководителем бригады». Диспетчер звонит из карточки своей службы и докладывает; начальник переспрашивает о том, чего не услышал, — не больше трёх вопросов — и говорит «Принял». Говорить можно голосом или набирать текстом: без сервера речи звонок целиком идёт текстом.

Что ждут в докладе, в материалах не сказано. Наше допущение (`OPEN-QUESTIONS.md`, вопрос 22) — то, что диспетчер знает сам: что случилось, где (улица и дом), пострадавшие, кого ещё оповестили, какую бригаду направил и на каком она этапе по последнему полученному докладу. Начальник ничего сверх карточки не спрашивает и не сообщает.

Разбор доклада — отдельный блок в разборе попытки ДДС: по каждому сведению «доложено», «не доложено» или «доложено неверно» (например, «пострадавших нет», когда в карточке двое), с дословной фразой диспетчера. Балл правил бригады он не меняет, решение по нему — за преподавателем. Устройство то же, что у анализа ручного ввода (раздел 3.9): доложенное отмечает модель, но только с цитатой, и цитату проверяет код; без модели доложенное ищут ключевые слова. Вопрос модели с подсказкой («Пострадавших двое?» — цифра, слово из адреса) заменяется готовой формулировкой: начальник спрашивает, а не подсказывает.

На процессоре реплика модели — 17–48 с, в телефонном разговоре это недопустимо. Поэтому по умолчанию начальник переспрашивает по ключевым словам без паузы, а модель один раз разбирает доклад после «Принял», в фоне (около 20 с; до этого в разборе — ключевые слова с пометкой). Настройка `chief.live_model` включает модель в каждую реплику — для сервера с видеокартой. Замер — `backend/docs/замеры/доклад-начальнику.md`.

3.5. Прогноз результата и его проверка

Критерий экспертизы просит не прогноз, а **его проверку** — сравнение прогнозных значений с реальными. Поэтому устройство такое:

1. **Прогноз фиксируется до начала вызова** и задним числом не переписывается; факт проставляется после разбора. Точность показывается только по фактически сделанным прогнозам.
2. **Модель — гребневая регрессия**, а не нейросеть: данных мало, сложная модель переобучится, а на разборе нужно показать, из чего сложился прогноз. Признаки — по последним разборам курсанта: средний балл, тренд, доля угаданного, доля превышений норматива, опыт, средний балл на той же группе классификатора. Коэффициенты считаются решением нормальных уравнений, без внешних библиотек.
3. **Рядом всегда базовая линия** — средний балл курсанта. Пока пар «прогноз — факт» меньше 30, модель не обучается и прогноз делает базовая линия; при истории меньше 3 разборов прогноз помечается недостоверным.
4. Достоверность — средняя абсолютная ошибка, доля попаданий в допуск (по умолчанию ±10 баллов) и та же ошибка у базовой линии: видно, стоит ли модель своих сложностей.

Курсант видит свой прогноз с разложением по признакам в разборе, преподаватель — достоверность прогноза по группе в аналитике.

3.6. Рекомендации курсанту и проверка их исполнения

Рекомендация сама по себе — декларация, поэтому проверяется так же, как прогноз. После разбора система ищет **повторяющиеся** ошибки: одно и то же поле с тем же видом замечания в последних разборах. Текст рекомендации собирается только из замечаний разбора — подпись поля плюс заготовка по виду ошибки, — модель его не пишет.

Рекомендация фиксируется с временем. В первом следующем разборе, где ошибка могла повториться, ставится вердикт: ушла или нет. Доля сработавших — измеренная величина, и рядом базовая линия: как часто та же повторяющаяся ошибка уходит сама, без рекомендации.

Пороги — **наше допущение** (`OPEN-QUESTIONS.md`, вопрос 19): окно 5 разборов, повтор не меньше 2 раз, открытых рекомендаций не больше 3, о доле судим не раньше 5 проверок. Сравнение с базовой линией строже к рекомендациям, чем к ней самой: рекомендации достаются самым частым ошибкам. На экране это сказано.

3.7. Языковая модель — только текст

Модель отвечает за то, что нельзя посчитать: черновики учебных вызовов (первая фраза заявителя, его ответы на вопросы), правку вызова по переписке с преподавателем, словесные сводки по группе. **Эталон для оценки она не считает никогда** — его считает классификатор. Ответы заявителя подготовлены заранее и выверены преподавателем — пока курсант ведёт вызов, модель их не сочиняет. Одно исключение — свободный вопрос заявителю (раздел 3.11): модель понимает, про какой из заготовленных фактов спросили, и выбирает его из закрытого списка; ответ заявителя и тогда — записанный в сценарии. После сохранения модель ещё в фоновой очереди читает свободный текст курсанта (раздел 3.9). Разбор правил курсант видит сразу, анализ модели приходит следом.

При подготовке учебного вызова конвейер устроен так:

1. признаки выбираются из **закрытого словаря** опросной карты;
2. ответы, которых нет в словаре, отбрасываются;
3. классификатор считает эталон;
4. если он жалуется (например, «уточните признак второго уровня»), его жалоба возвращается модели как замечание, и шаг повторяется;
5. только после этого модель пишет текст легенды и скрытых фактов.

Ответ модели ограничен **схемой JSON на уровне декодирования** (`response_format: json_schema`): движок отбрасывает токены, нарушающие схему, поэтому структурно неверный ответ невозможен.

Где работает модель. Сервер приложения обращается к ней по протоколу, совместимому с OpenAI API; адрес задаётся переменной `ML_BASE_URL`. В поставке это локальный llama.cpp на той же машине — внешних обращений нет, данные никуда не уходят. Тот же адрес можно направить на другой совместимый сервер (vLLM, Ollama) во внутреннем контуре, не меняя кода. При развёртывании через

`deploy/up.sh` достаточно указать `ML_BASE_URL` в `deploy/.env` — тогда свой контейнер модели не поднимается.

3.8. Исходные данные

Все данные, на которых работает тренажёр, взяты из материалов задачи и ответов экспертов. Их происхождение построчно зафиксировано в `PROVENANCE.md`:

- **ЕКП** — файлы классификатора двух версий (v046_11 от 15.11.2024 и v046_24 с корректировкой МВД и Департамента), импортированы целиком;
- **учебные вызовы** — 96 обращений, разобранных из 32 бумажных билетов; у всех 96 разобран телефон, у 57 — ФИО заявителя, у 25 — скрытый адрес. Из них готовятся учебные вызовы (раздел 4.6);
- **интерфейс** — снят по скриншотам боевой системы, сопоставление кадров описано в `frontend/SCENARIOS.md`; полный список типов происшествия, ветки карты 101 и справочник служб — по материалам, переданным экспертами в чате задачи.

Где данных заказчика не хватило, параметр вынесен в настройки и помечен как наше допущение (`PROVENANCE.md`, «Наши допущения»). Синтетических данных, выданных за данные заказчика, в поставке нет.

3.9. Нейросетевой анализ ручного ввода и экспертная оценка

ТЗ требует оценки «в том числе на основе анализа ручного ввода текста», «на основе нейросетевого анализа» и просмотра результатов как «автоматическая оценка И экспертная оценка». Всё, что проверяется сравнением, — признаки, адрес, службы, статусы, время — проверяют правила (разделы 3.3, 3.4). Правила не умеют читать свободный текст; это делает модель.

Что читает модель.

Попытка	Текст курсанта	С чем сверяется
Оператор 112	«Описание со слов заявителя», описательный адрес	реплики заявителя из стенограммы вызова
Диспетчер ДДС	комментарии к статусам реагирования	описание из карточки 112 и полученные доклады бригады

Анализ ставится в очередь при сохранении карточки 112 и при закрывающем статусе ДДС («Работы завершены», «Отказ от выполнения работ», «Не принята»).

Как устроен запрос. Модель не ищет расхождения сама — модель такого размера их пропускает: на пробном вызове она не заметила «горит баня» вместо «горит крыша дома». Вместо этого она разбирает текст курсанта по утверждениям и про каждое отвечает: подтверждается, противоречит или заявитель этого не сообщал; отдельно — что существенное заявитель сказал, а в тексте нет. Вид замечания из вердикта выводит код. Ответ ограничен схемой JSON на уровне декодирования.

Каждое замечание подтверждено цитатой, и цитата проверяется кодом. Фрагмент текста курсанта ищется в его тексте, фрагмент услышанного — в стенограмме или докладах (без учёта

регистра, «ё» и знаков препинания). Не нашлась цитата — замечание отбрасывается, и число отброшенных показывается преподавателю. Так модель не может приписать курсанту слова, которых он не писал, или заявителю — которых тот не говорил.

Балл модель не меняет. Замечания стоят в разборе отдельным блоком «Анализ текста (нейросеть)» со статусом (готовится, готов, модель недоступна), рядом с баллом правил. Преподаватель подтверждает или отклоняет каждое замечание. Модели нет (`ML_PROVIDER=mock`) — анализ честно помечается «модель недоступна», а не выдаёт пустой список, похожий на «замечаний нет». Подняли модель — преподаватель повторяет анализ кнопкой, курсанту проходить вызов заново не нужно.

Экспертная оценка. В карточке, открытой из журнала, преподаватель видит все попытки по ней — вызов 112 и работу каждой службы ДДС — и ставит экспертный балл 0–100 с комментарием. Автоматический балл при этом не переписывается: оба хранятся и показываются рядом, курсант видит оба и комментарий. Каждая оценка и её изменение пишутся в журнал действий (`grade.override`) вместе с автоматическим и прежним баллом, вердикты по замечаниям модели — тоже (`analysis.verdict`). Экспертный балл есть в отчёте по занятию (JSON и CSV) и в отчёте по группе. Администратор экспертную оценку не ставит: ТЗ запрещает ему вмешиваться в оценки, и мы не даём ему этого вовсе.

Проверено. Код вокруг модели — очередь, проверка цитат, хранение, права, аудит — покрыт проверками с подставной моделью. На настоящей модели (GigaChat 3.1 10B-A1.8B, llama.cpp, 6 потоков) проверено на нарочно испорченном описании вызова «горит крыша частного дома»: «Горит баня на участке, есть пострадавший. Дом газифицирован.» Модель нашла все четыре расхождения — баню вместо крыши дома, пострадавшего при «пострадавших нет», газификацию, о которой заявитель не говорил, и незаписанное «пламя видно». Ответ занимает **26–33 секунды** на процессоре. **Чего не проверяли:** согласие модели с экспертами на выборке вызовов — такой выборки у нас нет. Вид замечания («не записано» или «противоречие») модель от прогона к прогону путает, суть — нет; поэтому решение по каждому замечанию за преподавателем.

На процессоре описание 112 сверяет кросс-энкодер. Генеративная модель на процессоре не только медленна, но и плохо видит упущенное: на проверочном наборе (66 пар «описание — факт», три учебных вызова) GigaChat в 16 случаях из 26 назвал подтверждённым факт, о котором описание молчит. Кросс-энкодер OpenJEV решает ту же задачу как NLI — описание посылкой, факт вызова гипотезой, ответ «подтверждает / противоречит / молчит» — одним проходом, без генерации, одинаково при повторе. Сервер — `nli/server.py`, адрес `NLI_BASE_URL`; нет его — сверяет языковая модель, как описано выше. Выбор — настройка `analysis.engine` (авто, только кросс-энкодер, только языковая модель).

Кросс-энкодеру нужны факты утверждениями: «Нет, дом не газифицирован.» он читает как отрицание («Нет,»), и точность на сырых ответах заметно ниже. Поэтому при подготовке вызова языковая модель в очереди переписывает ответы заявителя и первую фразу утверждениями («Дом не газифицирован»), а преподаватель видит и правит их в разделе подготовки вызовов, рядом с ответами. Сверяются только факты, которые курсант слышал: легенда и ответы на заданные им вопросы.

сверка на процессоре	верно	ложных «противоречит»	с на описание
OpenJEV 4B, утверждения готовит GigaChat	45 из 49 (92 %)	1	3,4
OpenJEV 2B, утверждения готовит GigaChat	39 из 49 (80 %)	5	1,4
GigaChat 3.1 (66 пар, та же постановка)	48 из 66 (73 %)	1	10,4

Чекпоинт по умолчанию — 2B: 9 ГБ памяти против 18 ГБ у 4B, а на машине ТЗ (32 ГБ) рядом ещё языковая модель и база. Где память есть, 4B точнее: ложных «противоречит» впятеро меньше. «Добавлено от себя» кросс-энкодер не ищет — это обратное направление, на наборе оно не мерилось, и это остаётся языковой модели. Комментарии диспетчера ДДС сверять с фактами нечего — их всегда читает языковая модель. Подробно — `backend/docs/замеры/ОПИСАНИЯ.md`.

3.10. Голос: распознавание и синтез на процессоре

По ответу заказчика (чат, 27.09.2026) голосом диспетчер ДДС говорит со старшим бригады и с вышестоящим начальником; физический IP-телефон не обязателен, достаточно программной эмуляции. Голос заявителя 112 на этапе MVP исключён, «но вы можете его сделать — это будет плюсом» — сделан, раздел 3.11.

Что сделано. Разговор диспетчера со старшим бригады:

1. Диспетчер нажимает «Сказать голосом» и говорит в микрофон; браузер пересчитывает запись в WAV 16 кГц моно.
2. Реплика распознаётся (`POST /api/voice/asr`) и уходит вместе со звонком старшему; сказанное сохраняется в разговоре — преподаватель видит не только факт звонка, но и что курсант сказал.
3. Ответ старшего и его входящие доклады озвучиваются (`POST /api/voice/tts`), текст остаётся на экране. Звук можно выключить переключателем в панели бригады.

Модели. Распознавание — GigaAM v3 e2e RNNT (сразу с пунктуацией), синтез — TeraTTSv2 с готовыми голосами из поставки модели: старший бригады и начальник звучат разными голосами. Никого не клонируем. Обе модели в ONNX, работают на процессоре, видеокарта не нужна.

Что говорит собеседник, решает движок тренажёра, а не модель: текст доклада называет только этап работ (раздел 3.4). Модель голоса лишь произносит его. Поэтому голос не добавляет выдуманной обстановки и не меняет разбор: оценка ДДС по-прежнему считается по тому, когда диспетчер узнал об этапе и когда поставил статус.

Скорость (замер — `backend/docs/замеры/ГОЛОС.md`, 4 потока, машина под нагрузкой): синтез фразы доклада — 0,5–0,7 с на 3–5 с звука, распознавание реплики — 0,2–0,3 с. Повторная фраза берётся из кэша.

Без голоса всё работает. Сервер голоса — отдельный процесс; не подключён — `GET /api/voice/status` отвечает «нет», кнопки «Сказать голосом» нет, разговор идёт текстом, как раньше.

Чего не сделано. Точность распознавания на живой речи в шуме не мерили — только на синтезированных фразах.

3.11. Разговор оператора 112 с заявителем: своими словами и голосом

Заказчик (чат, 27.09.2026): «Диалог с заявителем исключён на этапе MVP, но вы можете его сделать — это будет плюсом». Сделан.

Как спрашивает курсант. В панели обучающегося два режима: «Из списка» — готовые вопросы кнопками, как раньше, и «Своими словами» — поле ввода и кнопка «Голосом». Сказанное распознаётся тем же сервером голоса (раздел 3.10) и уходит как напечатанный вопрос.

Как тренажёр понимает вопрос. Задача закрытая: вопрос либо про один из скрытых фактов вызова, либо ни про какой. Модель не сочиняет ответ — она выбирает факт из перечисления, заданного схемой JSON (`LabelScorer`, `POST /api/sessions/{id}/ask-free`). Заявитель отвечает записанным в сценарии; понятый вопрос засчитывается как выясненный, ровно как нажатая кнопка, и разбор «выяснил или угадал» не меняется. Вопрос ни про что — заявитель переспрашивает («Не поняла вас, повторите») и ничего не выдумывает. В стенограмме — дословно то, что сказал курсант: по ней видно, как он спрашивал. Нет модели — вопрос понимается по совпадению слов; хуже, но разговор не встаёт.

Голос заявителя. Первая фраза и ответы звучат голосом заявителя — мужским или женским по отчеству из сценария (стили TeraTTSv2 из поставки модели, `ru_m1` и `ru_f1`). Переключатель «звук» — в шапке панели. Сервера голоса нет — всё текстом, как раньше.

Замер (`backend/docs/замеры/СВОБОДНЫЙ-ВОПРОС.md`, 34 вопроса на двух вызовах, 7 — нарочно ни про какой факт):

модель	понят верно	«нечего отвечать»	медиана
ЦП: GigaChat 3.1, 6 потоков	27 из 27	5 из 7	1,74 с
GPU: Qwen3.8-27B, RTX 4090	27 из 27	6 из 7	0,35 с

На процессоре вопрос не по делу в двух случаях из семи получает ответ на ближайший факт, и этот факт засчитывается как выясненный, хотя спросили не про него. В стенограмме это видно: там дословный вопрос курсанта рядом с ответом заявителя, и преподаватель может поправить оценку экспертной (раздел 3.9). Синтез ответа заявителя — 0,4–0,5 с, распознавание вопроса — около 0,5 с.

Живая речь заявителя (профиль GPU). С настройкой `applicant.live_model = 1` заявитель отвечает не заготовленной фразой, а репликой модели — своими словами и с волнением: «Ой, да там уже пламя! Крыша горит открыто, огонь прямо видно!» вместо «Пламя видно, крыша горит открыто.». Какой факт раскрыт, по-прежнему решает сопоставление вопроса, и разбор от модели не зависит. Модели уходят только легенда, уже раскрытые ответы и ответ на текущий вопрос — ответов на незаданные вопросы в запросе нет. Реплика проверяется до показа: числа в ней должны быть из сценария (кодом), а второй запрос к модели подтверждает, что суть ответа передана и ничего не добавлено. Не прошла или не успела за 6 с — звучит заготовленная фраза, а отбракованная реплика с причиной пишется в журнал действий (`applicant.fallback`). Кнопочный режим тоже идёт через живую реплику.

Замер на трёх утверждённых вызовах, 19 вопросов (`backend/docs/замеры/ЖИВОЙ-ЗАЯВИТЕЛЬ.md`): на Qwen3.8-27B (RTX 4090) — 17 живых реплик из 19, ответ целиком 1,6 с (максимум 2,1 с); одна

отбраковка — перестраховка проверки, вторая поймала расхождение в самом сценарии (ФИО заявителя в карточке и в ответе на вопрос об имени разные). Проверка пропустила одну мелкую добавку («только дым»). На процессоре режим непригоден: реплика GigaChat — 8,3 с без проверки, все 19 ушли в заготовку по таймауту, поэтому по умолчанию он выключен. Настройка меняется через `PATCH /api/settings/applicant.live_model`.

Чего не сделано. Уточняющие вопросы, на которые у заявителя в сценарии нет факта («на каком этаже?», если этажа в сценарии нет), получают «не понял» (в живой речи — «не знаю, при чём тут это»), а не правдоподобный ответ: выдумывать сведения заявителю модель не даём.

---

4. Функциональная архитектура

4.1. Роли

Роль	Доступ
Курсант — оператор 112	учебные вызовы (билеты и задания занятия), приём вызова и заполнение карточки 112, свои карточки в журнале, разбор, прогноз, рекомендации, свои автоматическая и экспертная оценки
Курсант — диспетчер ДДС	лента карточек своей службы, карточка ДДС (поля 112 — только чтение), статусы реагирования, бригада и доклады, доклад начальнику, свой разбор ДДС с автоматической и экспертной оценкой
Преподаватель	занятия, подготовка вызовов, группы, аналитика по группе, экспертная оценка попыток и вердикты по замечаниям модели
Администратор	версии классификатора, учётные записи (заведение, отключение, удаление с резервной копией), настройки, системные журналы; оценки видит, но не ставит

Разграничение проверяется на сервере при каждом запросе; интерфейс скрывает недоступные разделы, но не является средством защиты.

Два рабочих места курсанта. Организатор, чат 18.09: «один интерфейс, одна учётка, две мини-роли — либо пользователь заносит данные от потерпевшего, либо пользователь передаёт карточку в ДДС и там её уже принимают». У учётной записи курсанта — профиль рабочего места (`users.duty`): **оператор 112** или **диспетчер ДДС** своей службы. Интерфейс общий, функционал разный:

Действие	Оператор 112	Диспетчер ДДС
билеты и задания для приёма вызова, начать вызов	да	нет
вопросы заявителю (кнопкой, своими словами, голосом)	да	нет
заполнить и сохранить карточку 112	да	нет
свои карточки в журнале, их разбор, прогноз, рекомендации	да	нет
лента карточек своей службы	нет	да
отметка открытия, статусы реагирования	нет	да
бригада, звонок старшему, доклады	нет	да

Действие	Оператор 112	Диспетчер ДДС
доклад вышестоящему начальнику	нет	да

Нарушение — отказ `403 wrong_duty` с объяснением. Профиль сервер берёт из базы на каждом запросе и при подписке на канал событий, поэтому смена профиля действует сразу, без перевхода. Незаконченный вызов 112, начатый до перевода в ДДС, продолжить нельзя: и экран, и сервер отвечают отказом. Прямая ссылка на экран другого рабочего места показывает «Недоступно для вашего профиля», а не пустоту. В шапке у курсанта — его рабочее место («Оператор 112» или «Диспетчер ДДС · служба»).

Профиль назначают администратор (любому курсанту, в разделе «Пользователи») и преподаватель (своим курсантам, в списке группы на странице занятий): обучающегося «могут направить на практику в одно или другое место». Диспетчеру ДДС служба обязательна — по ней отбирается его лента; оператору 112 она не нужна. Каждая смена — в журнале действий (`user.duty`). Занятие ведётся на одном рабочем месте: вызовы 112 достаются только операторам группы, карточки ДДС — только диспетчерам по их службам; нет в группе курсантов нужного профиля — занятие не запускается и говорит почему.

4.2. Оператор 112

```
вход → журнал происшествий → учебный вызов
                                  |
              разговор с заявителем ←→ заполнение карточки 112
                                  |
                        классификация → список оповещения
                                  |
                   разбор: оценка, прогноз, рекомендации
```

Курсант задаёт заявителю вопросы; ответ помечает факт выясненным — на этом держится разбор «узнал или угадал». Ввод в карточку уходит на сервер пакетами; правки в пределах одного окна **сливаются вглубь**, а не затирают друг друга. При обрыве сети пакет повторяется.

4.3. Диспетчер ДДС

Сохранённая оператором 112 карточка приходит в журнал каждой службы из списка оповещения. Диспетчер работает с несколькими карточками одновременно, как в текущей работе: пока он занят одной, время идёт и у ожидающих.

```
карточка в строке сообщений —(30 с)—► открыта —(3 мин)—► первая запись
      |                                                          |
      |                          «Принята» —► назначить бригаду
      |                                                  |
      |                  доклады старшего / звонок старшему
      |                                                  |
      └—► «Не принята» с причиной     статусы по факту —► «Работы завершены»
                                                         |
                                                  разбор ДДС
```

Звонок старшему и входящие доклады приходят в карточку ДДС текстом, а при подключённом голосе — ещё и голосом: диспетчер говорит в микрофон, ответы старшего озвучиваются (раздел 3.10). Рядом — звонок вышестоящему начальнику с докладом по карточке (раздел 3.4).

4.4. Занятие преподавателя

Порядок — из ТЗ (§10, «Проведение практического занятия»): преподаватель создаёт занятие для группы, выбирает режим и категории происшествий под тему, запускает, следит за ходом и завершает в любой момент.

Режим	Что получает группа
Приём вызова (112)	учебные вызовы по выбранным категориям; пока идёт занятие, курсант видит только их
Работа ДДС	карточки по службам курсантов — созданные системой, заполненные курсантами на занятиях 112 или вперемешку; до 5 карточек на службу, в случайном порядке

В режиме ДДС карточки занятия **поступают потоком**: с интервалом, который задаёт преподаватель при создании занятия, — а не все сразу. Ход занятия виден по каждому курсанту: сколько взято, сколько доведено до разбора, средний балл. При завершении недоработанные вызовы закрываются.

Вид занятия. Кроме обычного, преподаватель собирает марафон или блиц — на том же механизме занятий (группа, запуск, ход, завершение), отличаются только правила:

Вид	Что это	Правила
Марафон	экзамен на 5–10 вызовов приёма 112	вызовы выбирает преподаватель (в своём порядке) или система по категориям; курсант идёт строго по порядку, у каждого вызова одна попытка; разбора, рекомендаций и балла между вызовами нет — они открываются после последнего вызова или по завершении занятия; обрыв и F5 продолжают тот же вызов
Блиц	серия из 3–10 коротких заданий на один сегмент карточки	сегмент — адрес, тип происшествия с опросной картой или службы; у ДДС — статусы по докладам бригады. Разбор — сразу после каждого задания, в балл идут только поля сегмента, время не оценивается

Новой оценки ни марафон, ни блиц не вводят: берётся тот же разбор (раздел 3.3), блиц лишь ограничивает его сегментом, марафон — откладывает показ. В базе разбор хранится полным, поэтому отчёты преподавателя видят карточку целиком.

Итоговый протокол (`GET /api/lessons/{id}/protocol`) — по каждому курсанту: каждый вызов, балл (в блице — по сегменту), время, грубые ошибки, экспертная оценка; средний балл и общее время. Грубыми считаем неверное значение в любом поле и пропущенную службу или пропущенный тип происшествия — от них зависит, приедет ли нужная служба. Это наше допущение: в материалах задачи грубые ошибки не перечислены. Преподаватель видит протокол сразу и из него открывает карточку каждого вызова с разбором и экспертной оценкой; курсант — только свою строку, в марафоне — после последнего вызова.

Допущения по видам занятия: марафон — только у приёма вызова (у ДДС карточки и так идут потоком); нижняя граница блица — 3 задания; «пострадавшие» относятся к сегменту «тип происшествия».

4.5. Учётные записи

Администратор заводит, отключает и удаляет учётные записи курсантов, преподавателей и администраторов. Курсанту при заведении задаётся рабочее место — оператор 112 или диспетчер ДДС со службой (раздел 4.1); менять его могут администратор и преподаватель своей группы. ТЗ запрещает удалять критически важные данные без резервного копирования, поэтому удаление устроено так:

- запись **без истории** (завели и ни разу не вошли) стирается сразу;
- запись **с историей** сначала копируется в таблицу `user_backups` (учётные данные, группы, попытки с баллами), затем обезличивается: ФИО и логин заменяются, вход закрыт, из групп человек убран. Оценки, карточки и журнал действий остаются читаемыми;
- свою запись и последнего администратора удалить нельзя. Каждое удаление пишется в журнал действий (`user.delete`).

4.6. Подготовка учебных вызовов

Преподаватель запускает подготовку вызовов из билетов (конвейер раздела 3.7) и доводит каждый перепиской с моделью: «поменяй ФИО», «добавь, что из мусоропровода идёт пламя», «сделай первую фразу короче». Модель отвечает репликой и правкой; правка применяется к вызову, эталон пересчитывается классификатором. В переписке показывается, что **фактически** изменилось, — по сравнению вызова до и после, а не со слов модели. Сыгранные факты не удаляются, а снимаются с учёта, чтобы не ломать уже пройденные разборы.

Рядом с ответами заявителя преподаватель видит **утверждения** — те же ответы, переписанные моделью в повествовательную форму («Дом не газифицирован» вместо «Нет, дом не газифицирован»). По ним кросс-энкодер сверяет описание курсанта (раздел 3.9). Утверждения можно поправить вручную или подготовить заново; правка ответа сбрасывает его утверждение, и модель готовит новое в очереди.

Готовый вызов преподаватель утверждает — до утверждения курсанты его не видят. **Преподаватель — конечный валидатор**, как и просит заказчик.

Как правки преподавателя влияют на модель — ровно так: переписка по вызову сохраняется, и последние ходы подаются модели в контекст при следующей правке того же вызова, поэтому «нет, короче» имеет смысл. Весов модели правки не меняют, и на подготовку других вызовов не переносятся. Дообучения модели на правках преподавателя в поставке нет — как это должно выглядеть, заказчик пока не определил.

4.7. Аналитика

Диаграммы (тепловая карта ошибок, динамика, воронка, виды замечаний) строятся из статистики занятий средствами SVG, без сторонних библиотек. Там же — достоверность прогноза и доля

сработавших рекомендаций по группе, рядом с базовыми линиями. Словесную сводку по группе формулирует модель — но **числа в неё подаются уже посчитанными**, в виде именованных агрегатов.

4.8. Настраиваемые параметры

Нормативы, параметры прогноза, рекомендаций и шкалы бригады хранятся в таблице настроек, а не в коде. Администратор меняет их через API (`PATCH /api/settings/{key}`); у каждого параметра есть допустимые пределы, значение вне их сервер отклоняет.

---

5. Компонентная архитектура

5.1. Сервер

```
api/        HTTP-обработчики, схема OpenAPI
domain/     классификатор, статусы, оценка, бригада, прогноз, рекомендации
ingest/     импорт классификатора и билетов, вывод опросных карт
reports/    отчёты, прогноз и рекомендации над данными базы
jobs/       фоновые задачи: сторож нормативов, доклады бригад, очередь модели
ml/         адаптеры моделей: HTTP-сервер модели и заглушка
```

Доменный слой не знает ни о HTTP, ни о моделях: классификатор, оценка, разбор ДДС, прогноз и рекомендации — чистые функции над данными. Поэтому все **352 серверные проверки** выполняются без поднятия модели; там, где важен ответ модели, её заменяет подставная.

Описание API машиночитаемо (OpenAPI, utoipa). Отдельная проверка читает список маршрутов из исходного кода и **падает при расхождении** со схемой.

5.2. Клиент

Слоистая раскладка (Feature-Sliced Design): `shared` → `entities` → `widgets` → `pages`. Состояние — zustand, стили — модули SCSS, маршрутизация — HashRouter (стенд отдаётся статикой без настройки переписывания путей).

Канал событий WebSocket с переподключением по возрастающей задержке. Канал — **ускорение, а не источник истины**: при его отказе интерфейс продолжает работать на обычных запросах.

5.3. Сервер модели

llama.cpp в режиме сервера, совместимого с OpenAI API. Подробности — `backend/docs/MODEL-SERVER.md`.

5.3a. Сервер сверки описания (кросс-энкодер)

`nli/server.py` — небольшой сервер на Python (FastAPI, PyTorch на процессоре): `POST /nli {premise, hypotheses}` → вероятности «противоречит / следует / молчит» по каждой гипотезе, `GET /health`. Сервер приложения ходит к нему по адресу `NLI_BASE_URL` через адаптер `ml/nli.rs`; клиенту он не виден. Чекпоинт — `NLI_MODEL_DIR` (`qwen3.5-2b-nli-v5` или `qwen3.5-4b-nli-v5` из `AlexWortega/openjev`). Считает в float32: bfloat16 на процессоре без его поддержки вчетверо медленнее, а экспорт гибридной модели в ONNX штатным экспортёром не проходит.

5.4. Сервер голоса

`voice/server.py` — небольшой сервер на Python (FastAPI, onnxruntime): `POST /asr` (WAV → текст), `POST /tts` (текст и роль собеседника → WAV), `GET /health`. Сервер приложения ходит к нему по адресу `VOICE_BASE_URL` через адаптеры `ml/voice.rs`; клиенту он не виден. Синтезированные фразы кэшируются на диске по хешу текста и голоса. Числа и сокращения адреса («№», «ул.», «д. 28») сервер разворачивает в слова сам: рантайм синтеза пропускал число перед точкой, и в адресе терялся номер дома.

---

6. Условия и ограничения

6.1. Целевой стенд

Процессор Intel Core i7 или Xeon, не менее 6 ядер; 32 ГБ оперативной памяти; **графический ускоритель не предусмотрен**. Это задано системными требованиями, и из этого следует весь выбор в разделе 3.

6.2. Что из этого следует

Модель на процессоре — узкое место. Выбрана **GigaChat 3.1 10B-A1.8B**: из десяти миллиардов параметров на каждом токене работают 1,8 миллиарда (архитектура со смесью экспертов, 64 эксперта, используются 4). Скорость на процессоре определяется активными параметрами, а не общим размером, поэтому модель отвечает как небольшая, а знает как крупная. Лицензия MIT.

Дополнительно включено **спекулятивное декодирование на блоке MTP** (multi-token prediction), встроенном в модель. Поддержки этого режима для данной архитектуры в llama.cpp не было — она добавлена двумя патчами, которые приложены к поставке (`backend/docs/mtp-*.patch`).

Измеренный результат на 6 потоках: **10,3 → 14,8 токенов в секунду, прирост 44 %**. Существенно, что `--spec-draft-n-max 1`: при значении по умолчанию 3 проверка отбрасывала большую часть черновика (доля принятых токенов 30 %), при 1 — принимается 73 %.

Оговорки к замеру. Мерили на AMD EPYC 7532 с **восемью каналами памяти**; ограничены были потоки (6), но не каналы. Генерация на процессоре упирается в пропускную способность памяти, а не в ядра, поэтому это **потолок, а не пол**: на настольном i7 с двухканальной памятью скорость будет ниже. На такой машине мы не мерили и цифру не называем (`OPEN-QUESTIONS.md`, вопрос 11).

Сравнивались два прогона при одинаковой нагрузке, а не серия: порядок величины достоверен, вторая значащая цифра — нет.

Скорость модели почти не влияет на нормативы ТЗ: пока курсант ведёт вызов, модель вызывается только на свободный вопрос заявителю — ответ в несколько токенов, медиана 1,7 с на процессоре (раздел 3.11); кнопки вопросов работают без неё. Подготовка вызовов, сводки и анализ текста после сохранения идут в фоновой очереди. Очередь одна: если группа из двадцати курсантов сохранит карточки разом, анализ последнего придёт через двадцать ответов модели. Разбор правил от этого не зависит — он приходит сразу.

6.2а. Профиль GPU — тот же тренажёр с моделью покрупнее

Если заказчик выделит видеокарту (в чате 23.09: «могут, если нужно будет»), меняется одна переменная — `ML_BASE_URL`. Сборка, база, интерфейс и все правила оценки те же. Проверено на одной RTX 4090 (24 ГБ) под vLLM 0.30, скрипт запуска и флаги — `deploy/gpu/start-vllm.sh`. Сравнивали две модели, обе с лицензией Apache 2.0:

- **Gemma 4 26B-A4B** — смесь экспертов, на токене работают около 4 млрд параметров; квантование INT4 из официального QAT-чекпоинта (`cyankiwi/gemma-4-26B-A4B-it-qat-AWQ-INT4`, 17 ГБ). **Выбрана по умолчанию**: втрое быстрее и не хуже по качеству.
- **Qwen3.8-27B** — плотная, INT4 W4A16 (`RedHatAI/Qwen3.8-27B-INT4`, 19,5 ГБ).

Замеры 28.09 на наших запросах, каждая модель одна на своей видеокарте:

задача	ЦП: GigaChat 3.1	GPU: Qwen3.8-27B	GPU: Gemma 4 26B-A4B
генерация, токенов в секунду	14,8	54	170
подготовка вызова по билету (`try_generate 3 1`)	94 с	7 с	3 с
анализ описания курсанта с цитатами	26–33 с	7,6 с	2,3 с
разбор доклада начальнику после «Принял»	20,8 с	5,4 с	1,3 с
сверка описания с фактами, 66 пар (`backend/docs/замеры/ОПИСАНИЯ.md`)	73 %, 10,4 с на описание	92 %, 1,1 с	94 %, 0,45 с, ложных «противоречит» 0
свободный вопрос заявителю: понято из 27 / отсеяно бессмысленных из 7	27 / 5, 1,7 с	27 / 6, 0,35 с	27 / 7, 0,13 с
живая реплика заявителя 112 (`backend/docs/замеры/ЖИВОЙ-ЗАЯВИТЕЛЬ.md`)	8,3 с — непригодно	17 из 19 живые, 1,6 с	16 из 19 живые, 0,5 с

Реплики Gemma ближе к тексту сценария (6 из 19 — дословно), у Qwen — живее, но Qwen однажды добавил от себя «огонь перекинулся на второй этаж», которого в сценарии нет. Отбраковки у обеих — на вопросах не по делу: проверка реплики требует «сути ответа» там, где ответа по сценарию нет (известный недочёт проверки).

На GPU реплики начальника и заявителя укладываются в секунды, поэтому там включается модель в разговоре (`chief.live_model = 1`, `applicant.live_model = 1`, раздел 3.11); на процессоре начальник

спрашивает по ключевым словам, модель разбирает доклад один раз в конце, а заявитель говорит заготовленными фразами сценария.

6.3. Требования ТЗ к производительности — измерено

Замер 28.09.2026 на release-сборке. Сервер тренажёра и PostgreSQL вместе ограничены **6 физическими ядрами и 32 ГБ** — как одна машина из требований ТЗ к серверу. Процессор — AMD EPYC 7532, **это не машина заказчика**: память у него восьмиканальная, у настольного i7 — нет. Языковая модель выключена: во время занятия курсанта она нужна только для свободного вопроса заявителю (раздел 3.11), нагрузочный прогон спрашивает кнопками. Методика, сырые данные и скрипт — `backend/docs/замеры/НАГРУЗКА.md`.

Требование ТЗ (§7)	Измерено	Условия
отклик интерфейса ≤ 2 с при нагрузке до 100 пользователей	максимум 0,69 с, p99 0,18 с; без пауз между действиями (1783 запроса/с) — максимум 0,61 с, p99 0,27 с; ошибок 0	80 курсантов и 20 преподавателей, 60 с, пауза 1 с между действиями
не менее 20 одновременных сессий	50 вызовов, начатых в одну секунду, — все дошли до разбора, по 0,62–0,64 с на вызов целиком; 80 разом — все, до 2,2 с на вызов, самый долгий запрос 0,82 с	вызов целиком: вход, начало, пять вопросов, правка, сохранение
сбой сети до 30 с — восстановление без потери данных	правки, внесённые без сети, в базе через 0,1 с после её возврата; канал событий открыт через 0,1 с; та же сессия сохраняется и разбирается	e2e-тест `frontend/e2e/tests/сбой-сети.spec.ts`, три прогона из трёх
запись в БД ≥ 100 операций/с	1516 правок карточки в секунду, 3234 изменённых строки в секунду; p99 правки 41 мс	50 параллельных потоков, 20 с
аналитический отчёт ≤ 30 с	отчёт по группе — 0,56–0,71 с; отчёт по вызову (JSON, CSV) — 7–11 мс	5380 карточек и 5330 разборов в базе
задержка VoIP ≤ 150 мс	не измерялась (SIP нет)	это не время ответа языковой модели. Телефонного канала (SIP) нет: голос — программная эмуляция, которую заказчик признал достаточной (27.09); реплика распознаётся за 0,2–0,3 с, ответ синтезируется за 0,5–0,7 с (раздел 3.10)

Что замер нашёл и что исправлено. Начало вызова обучало модель прогноза заново на всей истории: p99 доходил до 2,45 с — за нормативом. Теперь история читается одним запросом, модель обучается раз в минуту в фоне, p99 — 0,19 с. Журнал диспетчера считал отметки своей службы для всех карточек до выбора страницы: 135 мс на 8879 карточках и рост с базой; теперь 16 мс. Канал событий после обрыва в 30 с возвращался ещё примерно через 18 с — теперь переподключается сразу, как браузер сообщает о сети.

Чего замер не показывает. Текст сводки по группе пишет языковая модель — это время сюда не входит (около минуты, раздел 6.4); числовой отчёт от модели не зависит. Не мерились: модель под

нагрузкой, сбой сети при сотне пользователей, настольный процессор. Вопрос заявителю, заданный во время обрыва, не досылается сам: курсант повторяет его, когда сеть вернулась, — правки карточки при этом не теряются.

6.4. Ограничения решения

- **Голос — только со старшим бригады.** Диспетчер ДДС говорит со старшим голосом, ответы озвучиваются (раздел 3.10). Разговор с вышестоящим начальником голосом и голос заявителя 112 не сделаны: у оператора 112 вопросы заявителю выбираются из списка, ответы — текстом. Телефонии (SIP) нет — программная эмуляция в браузере.
- **Подбор служб только по типу происшествия.** По ответу заказчика службы в боевой системе подтягиваются ещё и по району обслуживания и подчинённости объекта (школа → ДДС департамента образования, управы, округа). Справочника «адрес → службы», по словам заказчика, нет в природе; территориальных получателей тренажёр не подбирает (`OPEN-QUESTIONS.md`, вопрос 15).
- **Нет сообщения ДДС в 112 об ошибке в карточке.** По ответу заказчика диспетчер узнаёт об ошибке из звонка бригады и сообщает в 112 по телефону. Эталонных примеров ошибок заказчик не дал, поэтому ошибки в карточки ДДС тренажёр не подкладывает.
- **Дообучения модели по правкам преподавателя нет** (раздел 4.6). Вердикты преподавателя по замечаниям модели (раздел 3.9) хранятся и попадают в отчёты, но на следующие ответы модели не влияют.
- **Нейросетевой анализ текста не сверен с экспертами на выборке.** Проверен на отдельных вызовах; вид замечания модель от прогона к прогону путает. Поэтому замечания модели балл не меняют, а решение по ним за преподавателем (раздел 3.9).
- **Какая версия классификатора эталонная** (v046_24 или v046_11), заказчик ещё не ответил. Активна v046_24; администратор может загрузить и активировать другую, разницу между версиями и затронутые учебные вызовы отдаёт API (`/api/classifier/versions/{a}/diff/{b}`, `…/impact`).
- **Сводка по группе считается около минуты** — числа берёт база, но формулирует их модель на процессоре.
- **Кнопки боевого интерфейса без действия.** Экран копирует боевую систему, и эти кнопки оставлены на своих местах, чтобы курсант учился на знакомом экране. Нажатие ничего не делает, подсказка сообщает «в тренажёре не реализовано», для экранного диктора они помечены недоступными. Это: отключение звонка, записи звонков и список SMS, подстановка номера из АОН (полоса звонка); отметки «Нет на месте / отказ от скорой» и «Нет доступа / заблокированные» (карточка 112); «просмотр» и «дополнение» в шапке карточки, отметки ЧС и ЧП (карточка ДДС); расширенный поиск по параметрам (журнал); карта там, где она не подключена к адресу. В оценку эти действия не входят.
- **Тренажёр не подключается к боевой ПОВ-112.** Он воспроизводит её интерфейс и логику по материалам задачи; обмена данными с действующей системой нет и не предполагается.
- **Часть решений принята при неполных исходных данных.** Все такие места перечислены в `OPEN-QUESTIONS.md` с указанием, что именно неизвестно и какое допущение сделано, — а не замаскированы.

---

7. Сборка, установка и запуск

Для стенда — раздел 7.7, одна команда. Разделы 7.1–7.6 — локальный запуск для разработки.

7.1. Что должно быть на машине

Компонент	Версия	Проверено на
Rust	см. примечание	1.98.1
Node.js	20.19+	20.20.0
PostgreSQL	14+	16 (в контейнере)
Docker	любая	для базы; необязателен

Нижняя граница версии Rust не проверялась: указана та, на которой решение собрано. Редакция языка — 2021.

7.2. База данных

```
docker run -d --name arm112-db \
  -e POSTGRES_USER=arm112 -e POSTGRES_PASSWORD=arm112 \
  -e POSTGRES_DB=arm112 -p 5432:5432 postgres:16
```

Схема, учебные учётные записи и выверенные учебные вызовы разворачиваются миграциями при первом запуске сервера.

7.3. Сервер приложения

```
cd backend
cp .env.example .env          # при необходимости поправить DATABASE_URL
cargo run --release           # http://localhost:8080, накатит миграции
```

Классификатор ставится отдельной командой (один раз), после чего сервер нужно перезапустить — опросные карты выводятся из классификатора при старте:

```
cargo run --release --bin import_classifier -- \
  ../task/Классификатор_происшествий_v_046_24*.xlsx "v046_24"
```

Описание API после запуска: `http://localhost:8080/api/docs`. Учётные записи: `admin`, `teacher1`, `student1` (оператор 112), `student2`…`student5` (диспетчеры ДДС разных служб), пароль у всех `arm112`.

7.4. Веб-клиент

```
cd frontend
npm run install:all
```

```
npm run build                 # сборка в apps/react/dist
npm run dev                   # разработка, http://localhost:5173
```

Собранный клиент — статические файлы, отдаётся любым веб-сервером.

7.5. Сервер модели

Подробная процедура, включая наложение патчей MTP и переквантование весов, — `backend/docs/MODEL-SERVER.md`. Кратко:

```
llama-server -m GigaChat3.1-10B-A1.8B-q4_K_M.gguf \
  --host 127.0.0.1 --port 8090 -t 6 -c 8192 \
  --spec-type draft-mtp --spec-draft-n-max 1
```

Сервер приложения подключается к нему переменными окружения:

```
ML_PROVIDER=http
ML_BASE_URL=http://127.0.0.1:8090
```

Без них сервер работает на заглушке: журнал, карточки, классификация, занятия, оценка, разбор ДДС, прогноз и рекомендации полностью функциональны; нет только подготовки и правки учебных вызовов моделью, словесной сводки по группе и нейросетевого анализа текста (он помечается «модель недоступна»).

7.5а. Сервер голоса

Необязателен: без него разговор со старшим бригады идёт текстом.

```
python3 -m venv .venv-voice && .venv-voice/bin/pip install -r voice/requirements.txt
# веса: istupakov/gigaam-v3-onnx (файлы v3_e2e_rnnt*) и TeraSpace/TeraTTSv2
# в каталог /путь/к/голосу — см. deploy/README.md, шаг 3а
cd voice && VOICE_MODELS_DIR=/путь/к/голосу VOICE_THREADS=4 \
  ../.venv-voice/bin/uvicorn server:app --host 127.0.0.1 --port 8091
```

Сервер приложения запускать с `VOICE_BASE_URL=http://127.0.0.1:8091`. Замер скорости — `voice/bench.py`.

7.5б. Сервер сверки описания

Необязателен: без него описание курсанта сверяет языковая модель.

```
python3 -m venv .venv-nli
.venv-nli/bin/pip install torch --index-url https://download.pytorch.org/whl/cpu
.venv-nli/bin/pip install -r nli/requirements.txt
# чекпоинт: AlexWortega/openjev, подкаталог qwen3.5-2b-nli-v5 — deploy/README.md, шаг
3а'
```

```
cd nli && NLI_MODEL_DIR=/путь/к/qwen3.5-2b-nli-v5 NLI_THREADS=6 \
  ../.venv-nli/bin/uvicorn server:app --host 127.0.0.1 --port 8093
```

Сервер приложения запускать с `NLI_BASE_URL=http://127.0.0.1:8093`. Замер скорости — `nli/bench.py`, точности — `backend/docs/замеры/ОПИСАНИЯ.md`.

7.6. Проверки

```
cd backend && cargo test            # 352 проверки
cd frontend/e2e && npx playwright install chromium   # один раз
cd frontend && npm test             # 72 сценария в браузере

# Анализ текста настоящей моделью: нужен поднятый сервер модели
cd backend && cargo test --test text_analysis -- --ignored --nocapture
```

Проверки работают на **отдельной базе** (`TEST_DATABASE_URL` в `backend/.env`), а браузерный прогон поднимает свой сервер и клиент на своих портах — рабочий стенд они не трогают. Подготовка тестовой базы — `CONTRIBUTING.md`.

7.7. Развёртывание стенда

Весь тренажёр поднимается на чистой ВМ одной командой через docker compose:

```
deploy/up.sh
```

Сервис	Что это	Наружу
`web`	nginx: собранный клиент, прокси `/api` и `/ws` на сервер	один порт (80)
`backend`	сервер тренажёра, release-сборка; ежесуточная копия базы	нет
`db`	PostgreSQL 16	нет
`model`	llama.cpp с патчем MTP и весами GigaChat	нет

При первом запуске скрипт создаёт `deploy/.env` со случайными паролем базы и ключом подписи токенов, сервер сам накатывает миграции и импортирует классификатор. Модель включается, только если найден файл весов; без него стенд работает на заглушке (раздел 7.5). Проверка стенда — `deploy/check.sh`, ничего не меняет.

Требования к ВМ: 6 ядер x86-64, 32 ГБ памяти с моделью (16 ГБ без неё), 40 ГБ диска, Ubuntu 22.04 или 24.04, видеокарта не нужна. Подробно — `deploy/README.md`.

---

8. Перечень использованных библиотек

Полные версии зафиксированы в `backend/Cargo.lock` и `frontend/apps/react/package-lock.json`.

8.1. Сервер — прямые зависимости

Библиотека	Версия	Лицензия	Назначение
tokio	1.53.1	MIT	асинхронная среда выполнения
axum	0.8.9	MIT	HTTP-сервер и маршрутизация
tower, tower-http	0.5.3, 0.6.11	MIT	промежуточные слои, статика, CORS
sqlx	0.8.6	MIT / Apache-2.0	доступ к PostgreSQL, миграции
serde, serde_json	1.0.229, 1.0.151	MIT / Apache-2.0	сериализация
argon2, password-hash	0.6.0, 0.6.1	MIT / Apache-2.0	хранение паролей
jsonwebtoken	11.1.0	MIT	токены доступа
reqwest	0.12.28	MIT / Apache-2.0	обращение к серверу модели
tokio-tungstenite	0.29.0	MIT	канал событий WebSocket
utoipa, utoipa-swagger-ui	5.5.0, 9.0.2	MIT / Apache-2.0	схема OpenAPI
calamine	0.36.1	MIT	чтение файлов классификатора
chrono	0.4.45	MIT / Apache-2.0	время и нормативы
uuid	1.26.1	Apache-2.0 / MIT	идентификаторы
tracing, tracing-subscriber	0.1.44, 0.3.23	MIT	журналирование
anyhow, thiserror	1.0.104, 2.0.20	MIT / Apache-2.0	обработка ошибок
rand	0.10.3	MIT / Apache-2.0	генерация значений
async-trait	0.1.92	MIT / Apache-2.0	трейты адаптеров моделей
dotenvy	0.15.7	MIT	чтение настроек
futures-util, http-body-util	0.3.34, 0.1.5	MIT	вспомогательные

Полный граф — **344 пакета**. Распределение лицензий: MIT и/или Apache-2.0 — 280, Unicode-3.0 — 18, остальные (BSD, ISC, Zlib, Unlicense, CDLA-Permissive) — 46. **Копилефт-лицензий, обязывающих раскрывать исходный код, в графе нет.**

8.2. Клиент — прямые зависимости

Библиотека	Версия	Лицензия	Назначение
react, react-dom	19.3.0	MIT	интерфейс
react-router-dom	7.18.4	MIT	маршрутизация
zustand	5.0.15	MIT	состояние
vite, @vitejs/plugin-react	8.3.0, 6.1.1	MIT	сборка
sass	1.104.1	MIT	стили
typescript	6.0.3	Apache-2.0	типизация
oxlint	1.83.0	MIT	статический анализ

Библиотека	Версия	Лицензия	Назначение
@types/*	—	MIT	описания типов

Диаграммы аналитики нарисованы средствами SVG — сторонней библиотеки для графиков в поставке нет.

В дереве сборки присутствует `lightningcss` 1.33.0 под **MPL-2.0** (преобразователь CSS внутри Vite). Это инструмент сборки: в поставляемый клиент его код не попадает, изменения в него не вносились, поэтому требования MPL о раскрытии изменённых файлов не возникают.

8.3. Проверки (в поставку не входят)

Playwright (Apache-2.0) — сценарии в браузере.

8.4. Модель и развёртывание

Компонент	Лицензия
GigaChat 3.1 10B-A1.8B	MIT
llama.cpp	MIT
GigaAM v3 (распознавание речи, ONNX `istupakov/gigaam-v3-onnx`)	MIT
onnx-asr, onnxruntime	MIT
TeraTTSv2 (синтез речи)	в карточке модели не указана; модуль ударений (RUAccent) — MIT
FastAPI, uvicorn (серверы голоса и сверки)	MIT, BSD-3-Clause
OpenJEV (кросс-энкодер сверки описания, `AlexWortega/openjev`)	MIT (в карточке модели)
PyTorch, transformers (сервер сверки)	BSD-3-Clause, Apache 2.0
PostgreSQL	PostgreSQL License
nginx	BSD-2-Clause

Лицензия MIT у весов модели существенна: она разрешает коммерческое использование, в отличие от распространённых среди русскоязычных моделей лицензий с оговоркой NonCommercial, неприменимых для государственного заказчика.

Два патча к llama.cpp (`backend/docs/mtp-1-converter.patch`, `mtp-2-runtime.patch`) написаны в рамках этой работы и распространяются на условиях лицензии llama.cpp.

---

9. Состав документации репозитория

Файл	О чём
`README.md`	обзор, быстрый старт
`deploy/README.md`	развёртывание стенда на ВМ, эксплуатация
`BACKEND.md`	схема базы, контракт API, домен
`INTEGRATION.md`	встраивание в стороннюю систему
`PROVENANCE.md`	происхождение каждого набора данных и наши допущения
`OPEN-QUESTIONS.md`	вопросы к заказчику и принятые допущения
`CHAT-FINDINGS.md`	что уточнено в чате задачи и как это отражено в решении
`CONTRIBUTING.md`	правила работы с кодом, тестовая база
`backend/docs/MODEL-SERVER.md`	сборка и запуск сервера модели, замеры
`frontend/README.md`	архитектура клиента
`frontend/SCENARIOS.md`	сопоставление с экранами боевой системы
`frontend/DEFECTS.md`	найденные проверками дефекты